

OneView — Step-by-Step Installation Guide

Audience: system administrators installing OneView on a clean Linux VPS, cloud VM, or personal server. No prior OneView knowledge is assumed.

Primary path: full **Docker Compose** stack (PostgreSQL, Redis, API, worker, Nginx, Mailpit, pgAdmin, RabbitMQ, Prometheus, Grafana, Loki).

Secondary path (optional): host Node.js for the React UI and/or Prisma migrate/seed against the Docker Postgres port.

Source of truth: when older docs disagree with this guide, prefer `docker-compose.yml`, `.env.example`, and `package.json` scripts. See [Do implementation notes](#).

Table of contents

- [System requirements](#)
- [Install required software](#)
- [Install Docker and Docker Compose](#)
- [Clone the repository](#)
- [Configure environment \(`.env` \)](#)
- [Install npm dependencies and build packages](#)
- [Build images and start Docker Compose](#)
- [PostgreSQL: migrate and seed](#)
- [Start the application UI](#)
- [Default URLs, ports, and credentials](#)
- [Verify every service](#)
- [Common installation errors](#)
- [Upgrade procedure](#)
- [Doc vs implementation notes](#)
- [Backup and restore](#)
- [Production deployment recommendations](#)
- [Optional: host API / native Postgres](#)

1. System requirements

Resource	Minimum (dev / small team)	Recommended
OS	Linux (Ubuntu 22.04+ / Debian 12+), Windows 10/11 with Docker Desktop, or macOS	Ubuntu 22.04 LTS or newer
CPU	2 vCPU	4+ vCPU
RAM	4 GB	8+ GB (monitoring + RabbitMQ add load)
Disk	20 GB free	40+ GB SSD
Network	Outbound HTTPS for image pulls and <code>npm install</code>	Static public IP + DNS for production
Accounts	Ability to run Docker (root or <code>docker</code> group)	Non-root user in <code>docker</code> group

Software the stack expects

Software	Purpose	Version used by project
Git	Clone the repo	Any recent
Docker Engine	Run containers	24+ (Compose V2 plugin)
Docker Compose	Multi-service stack	<code>docker compose v2+</code>
Node.js + npm	Host-side <code>npm install</code> , Prisma migrate/seed, Vite UI	Node.js 20+ (Dockerfile uses <code>node:20-alpine</code>)

Native PostgreSQL on the host is **not** required for the primary Docker path. Compose creates Postgres 16 inside Docker and publishes it on host port

2. Install required software

Commands below use **Ubuntu/Debian**. Adapt package managers on other distros. On Windows, install Git for Windows, Docker Desktop, and Node, their official installers, then use PowerShell or Git Bash from the repo root.

2.1 Update the system (Linux)

```
sudo apt update && sudo apt upgrade -y
```

Purpose: apply security patches and current package indexes.

Validate: command exits with code `0`.

2.2 Install Git

```
sudo apt install -y git
git --version
```

Purpose: clone the OneView repository.

Validate: prints a version string (e.g. `git version 2.x`).

2.3 Install Node.js 20+ and npm

Option A — NodeSource (recommended on Ubuntu):

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs
node -v
npm -v
```

Option B — official Node.js LTS installer from <https://nodejs.org> (Windows/macOS).

Purpose: run workspace install, build @oneview/* packages, Prisma CLI, and the Vite frontend.

Validate: `node -v` shows v20.x or higher; `npm -v` prints a version.

Node is still required on the **host** for migrate/seed and the UI even when API/DB run in Docker. The API image builds its own Node inside Dock

3. Install Docker and Docker Compose

3.1 Install Docker Engine (Ubuntu example)

Follow the current Docker docs for your OS: <https://docs.docker.com/engine/install/>.

Typical Ubuntu flow:

```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/u
$(. /etc/os-release && echo \"$VERSION_CODENAME\") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Purpose: install Docker Engine and the Compose v2 plugin (`docker compose`).

Validate:

```
sudo docker --version
sudo docker compose version
```

3.2 Allow your user to run Docker (optional but recommended)

```
sudo usermod -aG docker "$USER"
# Log out and back in (or newgrp docker), then:
docker run --rm hello-world
```

Purpose: run Compose without `sudo`.

Validate: `hello-world` prints a success message.

3.3 Windows / macOS

1. Install [Docker Desktop](#).
 2. Ensure Docker Desktop is **running**.
 3. Validate: `docker compose version` in PowerShell or Terminal.
-

4. Clone the repository

Replace the URL with your actual remote (GitHub/GitLab/internal).

```
cd ~
git clone <YOUR_ONEVIEW_GIT_URL> OneView
cd OneView
```

If you already have the files (USB copy, CI artifact):

```
cd /path/to/OneView
```

Purpose: obtain source, docker-compose.yml, Prisma migrations, and scripts.

Validate:

```
ls docker-compose.yml package.json .env.example apps/Dockerfile prisma/schema.prisma
```

All listed paths should exist.

5. Configure environment (.env)

5.1 Create .env from the template

Linux / macOS:

```
cp .env.example .env
```

Windows PowerShell:

```
Copy-Item .env.example .env
```

Purpose: local secrets and connection strings are not committed; .env.example is the checked-in template.

Validate: .env exists and is listed by `ls -a / dir`.

5.2 Review required variables

Open .env and confirm it matches the **Docker-first** defaults (as in .env.example):

```
# Frontend (Vite) – talk to API through Nginx
VITE_API_BASE_URL=http://localhost:8080/api/v1

# Database – host port 15432 → container 5432
DATABASE_URL="postgresql://admin:admin@127.0.0.1:15432/oneview?schema=public"

# API / Auth (used when running API on the host; Compose also injects its own values)
API_PORT=3001
JWT_SECRET=oneview-dev-jwt-secret-change-me
JWT_EXPIRES_SECONDS=28800
JWT_REFRESH_DAYS=7
HMAC_PEPPER=oneview-dev-pepper-change-me
CORS_ORIGIN=http://localhost:5173,http://127.0.0.1:5173,http://localhost:4173,http://127.0.0.1:4173

# Redis (host tools / optional host API)
REDIS_URL=redis://127.0.0.1:6379

# Storage
STORAGE_PROVIDER=filesystem
STORAGE_ROOT=./data/files

# Mail – Mailpit catcher in Compose
MAIL_DRY_RUN=false
MAIL_PROVIDER=smtp
MAIL_FROM=noreply@oneview.local
MAIL_SMTP_HOST=127.0.0.1
MAIL_SMTP_PORT=1025
MAIL_SMTP_SECURE=false
APP_PUBLIC_URL=http://127.0.0.1:5173
```

Variable	Purpose
VITE_API_BASE_URL	Browser calls to the Nest API via Nginx (: 8080)
DATABASE_URL	Prisma on the host connecting to Docker Postgres on 15432
JWT_SECRET / HMAC_PEPPER	Token signing and search hashing — change for production

Variable	Purpose
CORS_ORIGIN	Allowed browser origins for the API
Mail / APP_PUBLIC_URL	Forgot-PIN emails and reset links

Inside Docker, the API uses Compose-injected URLs (`postgres:5432`, `redis:6379`, `mailpit:1025`). Your host `.env` is mainly for Prisma CLI, Vite, host-run API.

Validate:

```
grep -E '^(DATABASE_URL|VITE_API_BASE_URL|JWT_SECRET)=' .env
```

6. Install npm dependencies and build packages

From the repository root:

```
npm install
```

Purpose: install root app + npm workspaces (`packages/*`, `apps/*`) and generate lockfile links.

Validate: ends without errors; `node_modules/` exists.

```
npm run packages:build
```

Purpose: compile platform packages (`@oneview/security`, `redis`, `storage`, `mail`, `auth`) that Nest apps depend on. The Docker image also run: build; running it on the host is required for host-side tooling and keeps local builds consistent.

Validate: each workspace with a `build` script completes successfully.

Equivalent shortcuts later:

Command	Purpose
<code>npm run packages:build</code>	Build all workspace packages with a build script
<code>npm run docker:up</code>	<code>docker compose up -d --build</code>
<code>npm run docker:down</code>	<code>docker compose down</code>
<code>npm run db:migrate:deploy</code>	<code>prisma migrate deploy</code>
<code>npm run db:seed</code>	Blank seed (masters + admin)
<code>npm run db:seed:demo</code>	Full demo employees/projects

7. Build images and start Docker Compose

7.1 PostgreSQL in Compose (automatic)

You do **not** install Postgres on the host for the primary path. The `postgres` service:

- Image: `postgres:16-alpine`
- User / password / DB: `admin / admin / oneview`
- Host mapping: `15432 → 5432` (avoids conflict with a local Windows Postgres on 5432/5433)
- Volumes: `oneview_pgdata`, `oneview_pgbackups`
- Healthcheck: `pg_isready -U admin -d oneview`

7.2 Build and start the stack

```
docker compose up -d --build
```

Or:

```
npm run docker:up
```

Purpose:

- Build `oneview-api` and `oneview-worker` from `apps/Dockerfile` (Node 20, Nest build).
- Pull/start Postgres, Redis, Nginx, Mailpit, pgAdmin, RabbitMQ, Prometheus, Grafana, Loki.
- Start services in dependency order (Postgres/Redis healthy before API).

First build can take several minutes (`npm ci` + package builds inside the image).

Validate:

```
docker compose ps
```

Expect containers such as `oneview-postgres`, `oneview-redis`, `oneview-api`, `oneview-worker`, `oneview-nginx`, `oneview-mailpit`, `oneview-oneview-rabbitmq`, `oneview-prometheus`, `oneview-grafana`, `oneview-loki` with state **running** / **healthy** (API may need ~40s `start_period`)

```
docker compose logs -f api --tail=50
```

Look for a log line that the API is listening (e.g. on port 3001). Press `Ctrl+C` to stop following logs.

7.3 Services started by Compose

Service	Container name	Role
postgres	oneview-postgres	Application database
redis	oneview-redis	Cache / queues
api	oneview-api	Nest REST API (/api/v1)
worker	oneview-worker	Background jobs (BullMQ)
nginx	oneview-nginx	Reverse proxy → API on host :8080
mailpit	oneview-mailpit	Dev SMTP + inbox UI
pgadmin	oneview-pgadmin	DB admin UI
rabbitmq	oneview-rabbitmq	Message broker + management UI
prometheus	oneview-prometheus	Metrics scrape
grafana	oneview-grafana	Dashboards
loki	oneview-loki	Log aggregation

Important: the API container does **not** publish port 3001 to the host. Use **Nginx on port 8080** (`http://localhost:8080/api/v1/...`). `Dir :3001` only works inside the Docker network or if you add a host port mapping yourself.

8. PostgreSQL: migrate and seed

Run these on the **host** after Postgres is healthy. They use `DATABASE_URL` from `.env` (`127.0.0.1:15432`).

8.1 Wait until Postgres accepts connections

```
docker exec oneview-postgres pg_isready -U admin -d oneview
```

Validate: accepting connections.

8.2 Apply migrations

```
npx prisma migrate deploy
```

Or: `npm run db:migrate:deploy`

Purpose: apply all committed SQL under `prisma/migrations/` to the empty Docker volume (safe for existing DBs too).

Validate: Prisma reports migrations applied (or already applied); no `P1001` connection errors.

8.3 Seed data

Blank seed (required masters + one admin login):

```
npm run db:seed
```

Purpose: create departments, skills, activities, app settings, and user `admin@acme.io` with PIN **12345** (Argon2 hash). No demo projects/extra emplc

Validate: console ends with something like:

```
Blank seed complete.  
Login: admin@acme.io / PIN 12345
```

Optional full demo dataset:

```
npm run db:seed:demo
```

Purpose: load demo employees, projects, and permissions (all demo PINs **12345**).

8.4 Optional: browse data

```
npm run db:studio
```

Opens Prisma Studio against `DATABASE_URL`.

Destructive reset (dev only):

```
npm run db:reset
```

Wipes the database, re-migrates, and re-runs the configured seed. Do **not** use on production data.

9. Start the application UI

The React SPA lives at the **repo root** and is not served by the Compose Nginx config (Nginx only proxies `/api/`). Start Vite on the host:

```
npm run dev
```

Purpose: run the OneView UI (default Vite port **5173**).

Validate: terminal shows a local URL; open <http://localhost:5173>.

Login with:

Field	Value
Email	admin@acme.io
PIN	12345

For a production-like static UI build (optional):

```
npm run build
npm run preview
```

Preview typically uses port **4173** (ensure it is listed in `CORS_ORIGIN` / API CORS).

Optional: host-run API (instead of / in addition to Docker API)

Only if you want hot-reload Nest development **without** using the `api` container:

1. Keep Postgres + Redis (and ideally Mailpit) up via Compose.
2. Stop the container API if ports/CORS conflict: `docker stop oneview-api` (and possibly `nginx`).
3. Build packages, then:

```
npm run packages:build
npm run api:dev
```

Point `VITE_API_BASE_URL` at `http://localhost:3001/api/v1` when talking to a host API directly.

Primary install path: keep the Docker API + Nginx and use `VITE_API_BASE_URL=http://localhost:8080/api/v1`.

10. Default URLs, ports, and credentials

10.1 Application endpoints

What	URL
UI (Vite)	<code>http://localhost:5173</code>
API via Nginx	<code>http://localhost:8080/api/v1</code>
API health	<code>http://localhost:8080/api/v1/health</code>
Nginx healthz	<code>http://localhost:8080/healthz</code>
OpenAPI / Swagger	<code>http://localhost:8080/api/docs</code>

10.2 Published host ports (from `docker-compose.yml`)

Port	Service
15432	PostgreSQL (host → container 5432)
6379	Redis
8080	Nginx → API
1025	Mailpit SMTP
8025	Mailpit web inbox
5050	pgAdmin
5672	RabbitMQ AMQP
5552	RabbitMQ Stream
15672	RabbitMQ Management UI
9090	Prometheus
3000	Grafana
3100	Loki

10.3 Default credentials (development only)

System	Username / email	Password / PIN
PostgreSQL	admin	admin (DB oneview)
App login (after seed)	admin@acme.io	PIN 12345
pgAdmin	admin@acme.io	admin
Grafana	admin	admin
RabbitMQ	admin	admin (vhost /)
JWT / HMAC defaults	see <code>.env.example</code>	change in production

Never use these passwords/PINs on a public production internet deployment without rotation and hardening (see [§16](#)).

10.4 pgAdmin: register the server

In pgAdmin (<http://localhost:5050>), add a server:

Field	Value
Host	postgres (from inside Docker) or <code>host.docker.internal</code> / <code>172.x</code> as needed; from another machine use the VM IP and port 15432
Port	5432 if connecting as <code>postgres</code> hostname on the Docker network; 15432 from the host OS
Username	admin
Password	admin
Database	oneview

10.5 Grafana → RabbitMQ (optional)

Field	Value
Host	rabbitmq (not localhost from inside Grafana)
AMQP / Stream	5672 / 5552
User / pass	admin / admin

Do not use RabbitMQ's default `guest / guest` from other containers (loopback-restricted).

11. Verify every service

Run these checks after install.

11.1 Container status

```
docker compose ps
```

All listed services should be **Up**. API/worker/postgres/redis/rabbitmq should eventually be **healthy** where healthchecks exist.

11.2 Nginx + API health

```
curl -s http://127.0.0.1:8080/healthz
curl -s http://127.0.0.1:8080/api/v1/health
```

Expected:

- /healthz → ok
- /api/v1/health → JSON with "status":"ok", "database":"up", "service":"oneview-api"

11.3 Postgres

```
docker exec oneview-postgres psql -U admin -d oneview -c "SELECT email, is_super_admin FROM employees ORDER BY email"
```

Expected: at least admin@acme.io after seed.

11.4 Redis

```
docker exec oneview-redis redis-cli ping
```

Expected: PONG

11.5 Login API smoke test

```
curl -s -X POST http://127.0.0.1:8080/api/v1/auth/login \
-H "Content-Type: application/json" \
-d '{"email":"admin@acme.io","pin":"12345"}'
```

Expected: JSON containing access/refresh tokens (not an auth error). Exact field names follow the live OpenAPI at /api/docs.

11.6 UI

1. Open http://localhost:5173
2. Sign in as admin@acme.io / 12345
3. Confirm cockpit or home loads without API connection errors (browser Network tab: calls to :8080/api/v1/... return 2xx)

11.7 Mailpit (forgot-PIN)

1. Open http://127.0.0.1:8025
2. Trigger forgot-PIN for a **registered** email (e.g. admin@acme.io)
3. Message appears in Mailpit when MAIL_DRY_RUN=false and SMTP points at Mailpit

11.8 Observability UIs

Check	URL
Prometheus	http://localhost:9090
Grafana	http://localhost:3000 (admin / admin)
RabbitMQ	http://localhost:15672 (admin / admin)
Loki	http://localhost:3100 (API; wire in Grafana as needed)

Acceptance checklist reference: [docs/acceptance-checklist.md](#).

12. Common installation errors

Problem	Likely cause	Solution
P1001 / can't reach database	Compose not up, wrong port, or Windows Postgres conflict	docker compose ps; use 127.0.0.1:15432 in DATABASE_URL; v healthy Postgres
connection refused on 5432	Docs that mention native :5432 while Docker publishes 15432	Use .env.example URL with 15432
Port 15432 / 8080 / 6379 already in use	Another process bound the port	Stop the other service or change the left-hand port in docker-comp and update .env
API health fails / nginx 502	API still starting or crashed	docker compose logs api; wait for start_period; rebuild doc compose up -d --build
curl :3001 fails from host	Port not published	Use http://127.0.0.1:8080/api/v1/...

Problem	Likely cause	Solution
<code>npm install</code> / build fails (argon2, native)	Wrong Node version or missing build tools	Use Node 20+; on Linux install <code>build-essential</code> / <code>python3</code> if native fail
Docker build OOM / killed	Insufficient RAM	Increase VM RAM to 4–8 GB; close other apps
Login fails after seed	Seed not run, or blank DB	Re-run <code>npm run db:seed</code> ; confirm row in <code>employees</code>
CORS errors in browser	UI origin not allowed	Add your UI origin to Compose <code>CORS_ORIGIN</code> / <code>.env</code> and recreate container
Forgot-PIN email missing	Unregistered email or dry-run	Use seeded email; check Mailpit; ensure API <code>MAIL_*</code> points at <code>mail</code> (Compose default)
Permission denied for Docker	User not in <code>docker</code> group	<code>sudo usermod -aG docker \$USER</code> and re-login
Windows: bash backup scripts fail	Scripts are bash	Use Git Bash/WSL, or run the <code>docker exec / docker cp</code> lines mar §15
pgAdmin can't reach DB	Wrong host/port from browser context	From host tools use <code>127.0.0.1:5432</code> ; from pgAdmin container use <code>postgres</code> port <code>5432</code>

13. Upgrade procedure

1. **Backup** the database first ([§15](#)).
2. Fetch new code:

```
cd /path/to/OneView
git pull
```

3. Refresh host dependencies and packages:

```
npm install
npm run packages:build
```

4. Rebuild and restart containers:

```
docker compose up -d --build
```

5. Apply new migrations:

```
npx prisma migrate deploy
```

6. Seed only if release notes require it (usually **do not** re-seed production; blank/demo seed may upsert or conflict depending on seed scripts):

```
# Dev only when you intentionally want demo/blank data refreshed
# npm run db:seed
```

7. Restart or rebuild the UI if running on the host:

```
npm run build # production static build
# or restart npm run dev
```

8. Re-run [§11](#) health checks.

Rollback: restore the previous DB dump, `git checkout` the previous tag/commit, and `docker compose up -d --build` that revision.

14. Doc vs implementation notes

These older doc snippets conflict with the **current** Compose + `.env.example` implementation. **Implementation wins.**

Topic	Older docs / README fragments	Current implementation
Hosting order	"Local Postgres first; Docker later" (<code>postgres-local-setup.md</code> , parts of README / architecture / skill)	Docker Compose is the primary stack ; host port
<code>DATABASE_URL</code>	<code>localhost:5432</code>	<code>127.0.0.1:5432</code> (see <code>.env.example</code> , <code>AGENTS.</code>
<code>VITE_API_BASE_URL</code>	Sometimes <code>http://localhost:3001/api</code> (missing <code>v1</code>)	<code>http://localhost:8080/api/v1</code>
API on host : <code>3001</code>	Implied as always available	Compose does not publish <code>3001</code> ; use Nginx 8080

Topic	Older docs / README fragments	Current implementation
Seed users	README lists several demo emails for UI	<code>npm run db:seed</code> = admin only ; use <code>npm run db:seed:demo</code> for full demo roster
Auth hashing	Some README text still says <code>bcrypt</code>	Seeds use Argon2 (<code>prisma/seed.ts</code>)
Architecture diagram	"API server future"	Nest API + worker apps exist under <code>apps/</code>

Native Postgres setup remains valid as an **optional** path: [docs/postgres-local-setup.md](#) — adjust ports carefully if Docker is also running.

15. Backup and restore

Scripts (Linux / macOS / Git Bash / WSL):

```
bash scripts/backup-postgres.sh ./backups
bash scripts/restore-postgres.sh ./backups/oneview_YYYYMMDD_HHMMSS.dump
```

What backup does

1. `pg_dump` custom format inside `oneview-postgres` to `/backups/...`
2. `docker cp` the file to the host directory you pass (default `./backups`)

What restore does

1. Copies the dump into the container
2. `pg_restore -c` into database `oneview` (cleans existing objects — **destructive**)

Manual equivalent (any OS with Docker):

```
# Backup
STAMP=$(date +%Y%m%d_%H%M%S)
docker exec oneview-postgres pg_dump -U admin -d oneview -F c -f /backups/oneview_${STAMP}.dump
docker cp oneview-postgres:/backups/oneview_${STAMP}.dump ./backups/oneview_${STAMP}.dump

# Restore
docker cp ./backups/oneview_YYYYMMDD_HHMMSS.dump oneview-postgres:/backups/restore.dump
docker exec oneview-postgres pg_restore -U admin -d oneview -c /backups/restore.dump
```

Named volumes to consider for full disaster recovery:

`oneview_pgdata`, `oneview_pgbackups`, `oneview_files`, `oneview_redis`, `oneview_rabbitmq`, plus Grafana/Prometheus/Loki/pgAdmin volume
DB dump covers application data; uploaded files live under the `oneview_files` volume (`STORAGE_ROOT` in API).

16. Production deployment recommendations

This Compose file is a **production-like local/dev stack**. Before exposing to the internet:

1. **Secrets:** set strong unique `JWT_SECRET`, `HMAC_PEPPER`, Postgres password, Grafana/pgAdmin/RabbitMQ passwords. Never commit `.env`.
2. **Postgres:** change `POSTGRES_PASSWORD` / role password; do not keep `admin/admin`. Restrict host bind — prefer Docker network only, or firewall is not public.
3. **TLS:** terminate HTTPS with a reverse proxy (Caddy, Traefik, or host Nginx) in front of `:8080` / the UI. Update `CORS_ORIGIN`, `APP_PUBLIC_URL`, `VITE_API_BASE_URL` to `https://` URLs.
4. **UI hosting:** build with `npm run build` and serve `dist/` via Nginx or a CDN; do not run `vite` in production.
5. **Mail:** replace Mailpit with a real SMTP provider; set `MAIL_SMTP_*` and `MAIL_DRY_RUN=false` carefully.
6. **PIN policy:** force users to change the demo PIN `12345`; limit exposure of seed accounts.
7. **Resources:** size RAM for Postgres + Redis + monitoring; disable Prometheus/Grafana/Loki/pgAdmin/Mailpit in hardened prod if unused.
8. **Backups:** schedule `scripts/backup-postgres.sh` (or equivalent) daily; test restore.
9. **Updates:** pin image tags; run migrations in a maintenance window; health-check before routing traffic.
10. **Access control:** SSH keys only; fail2ban/firewall; no public Redis/RabbitMQ/Postgres ports.

Also review [docs/acceptance-checklist.md](#) and [docs/docker-deployment.md](#).

17. Optional: host API / native Postgres

17.1 Docker DB + host UI (common hybrid)

Already covered: Compose for backend, `npm run dev` for UI, Prisma against `15432`.

17.2 Native PostgreSQL only

Follow [docs/postgres-local-setup.md](#):

- Install PostgreSQL 16 on the host
- Create role/db `admin/admin/oneview`

- Set DATABASE_URL to postgresql://admin:admin@localhost:5432/oneview?schema=public
- npx prisma migrate deploy && npm run db:seed

Stop Compose Postgres (or change ports) to avoid conflicts with Docker's **15432** mapping / a second instance.

17.3 Full host development APIs

```
npm install
npm run packages:build
# Postgres + Redis available (Docker or native)
npm run api:dev
npm run worker:dev # optional
npm run dev        # UI
```

Set .env DATABASE_URL / REDIS_URL / mail hosts to wherever those services listen.

Quick reference — minimal install sequence

```
git clone <YOUR_ONEVIEW_GIT_URL> OneView && cd OneView
cp .env.example .env
npm install
npm run packages:build
docker compose up -d --build
# wait until postgres healthy
npx prisma migrate deploy
npm run db:seed
npm run dev
```

Then open <http://localhost:5173> — login `admin@acme.io` / PIN 12345.
API: <http://localhost:8080/api/v1/health>

Related documentation

Doc	Purpose
docker-deployment.md	Short Compose quick start + port table
postgres-local-setup.md	Native Windows/Linux Postgres (optional)
monorepo.md	npm workspaces / @oneview/* packages
database.md	Schema conventions + seed variants
acceptance-checklist.md	Production acceptance checks
api-contract.md	HTTP API contract draft
Root README.md	Project overview
AGENTS.md	Agent / contributor locked choices